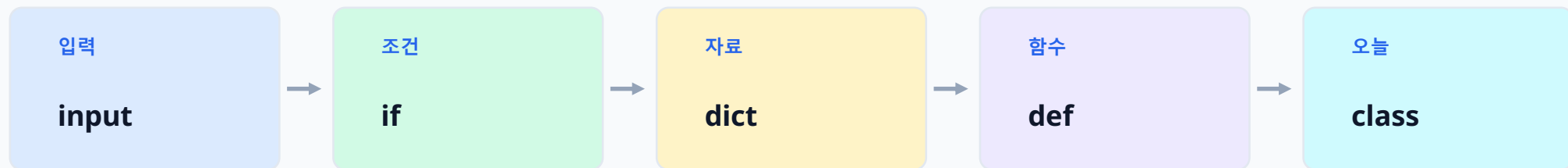


Chainlit으로 나만의 챗봇 만들기

함수에서 클래스로, 그리고 다음 시간 LLM 원리로 이어지는 수업

지난 시간에서 이어가기

오늘은 함수 다음 단계에서 출발합니다.



지난 시간에는 작은 기능을 직접 만들어봤습니다. 이번 시간에는 그 기능을 채팅 화면에 연결하고, 코드가 길어질 때 어떻게 정리하는지 배웁니다.

오늘의 목표

작동하는 챗봇 코드를 만들면서 클래스의 필요성을 이해합니다.

오늘 만들 것

키워드에 따라 답하는 챗봇

오늘 이해할 것

**함수로 시작한 코드를
클래스로 정의할 수 있다.**

오늘 사용할 파일

하나의 흐름으로 이어가겠습니다.

기본 파일

myChatbot.py

진행 방식

처음에는 이 파일을 계속 고치며 실습합니다.

Chainlit 설치하기

Chainlit은 기본 Python에 들어있는 기능이 아닙니다.

외부 라이브러리 가져와 설치하기

```
python -m pip install chainlit
```

설치하는 이유

채팅 화면을 만들어주는 외부 라이브러리를 가져오는 단계입니다.

설치가 끝난 뒤에는 같은 Codespaces 안에서 Chainlit 앱을 실행할 수 있습니다.

Chainlit 실행하기

VS Code 실행 버튼이 아니라 터미널 명령어로 실행합니다.

누르지 않기

VS Code 위쪽 실행 버튼



이렇게 실행

`python -m chainlit run test/test.py -w`

실행 후 PORTS 탭에서 8000번 포트를 열면 채팅 화면을 볼 수 있습니다.

```
python -m chainlit run myChatbot.py -w
```

1. Chainlit 화면에 연결하기

입력한 문장이 Python 코드로 들어오는 흐름부터 확인합니다.

첫 번째 Chainlit 코드

채팅창에 입력한 문장을 다시 보여줍니다.

```
import chainlit as cl

@cl.on_message
async def on_message(message: cl.Message):
    user_text = message.content

    await cl.Message(
        content=f"입력한 내용: {user_text}"
    ).send()
```

입력 문장

message.content

답장 보내기

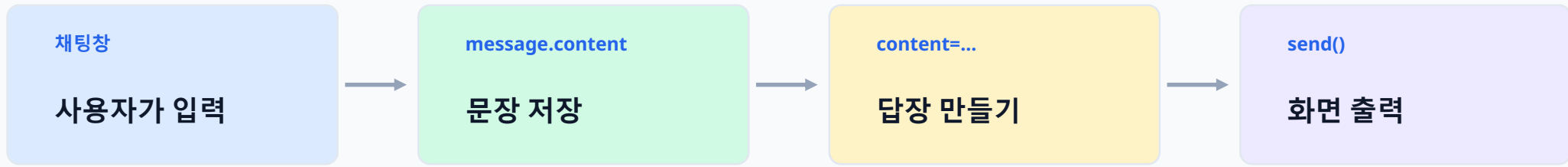
cl.Message(...).send()

실행

python -m chainlit run myChatbot.py -w

이 코드는 이렇게 움직입니다

화면과 Python 코드가 연결되는 순간입니다.



오늘의 모든 실습은 이 흐름 위에 쌓입니다. 먼저 화면을 만들고, 그 다음 답변 로직을 조금씩 바꿉니다.

해보기 1

화면에 나오는 문장을 바꿔봅니다.

1. content 안의 문장을 자기 팀 이름이 나오게 바꾸기
2. 채팅창에 아무 문장이나 입력하기
3. 화면에 바뀐 문장이 나오면 성공

```
content=f"[1팀 챗봇] 입력한 내용: {user_text}"
```

2. 답변 기능을 함수로 분리하기

화면 처리와 답변 처리를 나누면 다음 단계로 확장하기 쉽습니다.

답변 문장을 함수로 빼기

on_message 안에서 바로 답장을 만들지 않겠습니다.

answer 함수가 답변 문장을 만들도록 맡깁니다.

```
def answer(user_text):  
    return f"입력한 내용: {user_text}"
```

의미

사용자 입력을 받으면
답변 문장을 돌려주는 함수입니다.

아직 챗봇이 똑똑해진 것은 아닙니다. 대신 답변 만드는 부분을 따로 고칠 수 있게 되었습니다.

Chainlit과 함수 연결

전체 파일은 이렇게 이어집니다.

```
import chainlit as cl

def answer(user_text):
    return f"입력한 내용: {user_text}"

@cl.on_message
async def on_message(message: cl.Message):
    user_text = message.content
    reply = answer(user_text)

    await cl.Message(content=reply).send()
```

바뀐 부분

```
reply = answer(user_text)
```

역할 분리

**Chainlit은 화면 처리,
answer는 답변 처리**

조건을 넣으면 챗봇처럼 보입니다

지난 시간의 if 문을 답변 함수 안에서 사용합니다.

```
def answer(user_text):  
    if "공부" in user_text:  
        return "오늘 할 일을 작게 나눠보세요."  
    elif "진로" in user_text:  
        return "좋아하는 과목부터 적어보세요."  
    else:  
        return "조금 더 자세히 말해줄래요?"
```

[앞 코드와 연결](#)

방금 만든 **answer** 함수 안에 조건문을 추가한 것입니다.

이제 사용자가 입력한 문장에 “공부”나 “진로”가 들어 있으면 답변이 달라집니다.

해보기 2

챗봇이 알아듣는 단어를 늘립니다.

1. "친구"라는 단어가 들어오면 다른 답변하기
2. "운동" 또는 "급식" 같은 단어 하나 더 추가하기
3. else 문장은 마지막 안전망으로 남겨두기

```
elif "친구" in user_text:  
    return "상대 이야기를 먼저 들어보세요."
```

함수만 커지면 생기는 문제

키워드가 늘어날수록 answer 함수가 길어집니다.

```
def answer(user_text):  
    if "공부" in user_text:  
        return "공부 조언"  
    elif "진로" in user_text:  
        return "진로 조언"  
    elif "친구" in user_text:  
        return "친구 조언"  
    elif "운동" in user_text:  
        return "운동 조언"  
    else:  
        return "기본 답변"
```

문제

답변이 많아질수록
코드가 길어집니다.

다음 단계

답변 목록을
한 곳에 모읍니다.

답변 목록을 딕셔너리로 모으기

키워드와 답변을 한 곳에서 관리합니다.

```
answers = {  
    "공부": "오늘 할 일을 작게 나눠보세요.",  
    "진로": "좋아하는 과목부터 적어보세요.",  
    "친구": "상대 이야기를 먼저 들어보세요."  
}  
  
def answer(user_text):  
    for keyword, reply in answers.items():  
        if keyword in user_text:  
            return reply  
    return "조금 더 자세히 말해줄래요?"
```

좋아진 점

답변을 추가할 때 **answers**에 한 줄만 추가하면 됩니다.

연결

Chainlit 코드는 그대로 두고 **answer** 함수만 바꿉니다.

해보기 3

딕셔너리 답변 목록을 바꿔봅니다.

1. answers에 키워드 2개 추가하기
2. 추가한 키워드를 채팅창에서 직접 입력해보기
3. 없는 키워드를 입력했을 때 기본 답변도 확인하기

```
"급식": "오늘 메뉴를 확인해보면 좋겠어요.",  
"운동": "가볍게 10분부터 시작해보세요."
```

3. 클래스가 필요한 순간

답변 목록과 답변 기능을 하나의 챗봇 객체로 정리합니다.

클래스 용어를 정확히 잡기

공식 용어를 먼저 익힌 뒤, 같은 챗봇 코드 안에서 바로 확인합니다.

class / 클래스

객체의 구조와 동작
을 정해 둔 코드

object / 객체

클래스로 실제 만들어
진 대상

attribute / 속성

객체 안에 저장된
변수

method / 메서드

객체 안에 들어 있는
함수

예시로 연결해서 보기

```
class ChatBot:
    def __init__(self, name):
        self.name = name

    def answer(self, user_text):
        return print(self.name + ": 답해볼게요.")

bot = ChatBot("진로봇")
bot.answer("공부 도와줘")
```

class는 코드로 만든 설계도 이고,
object는 그 클래스로 실제 만든 대상입니다.

왜 클래스로 바꿀까?

따로 있는 정보와 기능을 하나로 모아 관리하기 위해서입니다.



딕셔너리와 함수가 따로 있어도 동작은 합니다. 하지만 챗봇 이름, 답변 목록, 말투, 질문 횟수처럼 챗봇이 관리해야 할 내용이 늘어나면 하나의 객체로 정리하는 편이 훨씬 읽기 쉽습니다.

첫 번째 클래스: AdviceBot

방금 만든 `answers`와 `answer` 함수를 클래스 안으로 옮깁니다.

```
class AdviceBot:
    def __init__(self):
        self.answers = {
            "공부": "오늘 할 일을 작게 나눠보세요.",
            "진로": "좋아하는 과목부터 적어보세요.",
            "친구": "상대 이야기를 먼저 들어보세요."
        }

    def answer(self, user_text):
        for keyword, reply in self.answers.items():
            if keyword in user_text:
                return reply
        return "조금 더 자세히 말해줄래요?"
```

바뀐 점

`answers`가 `self.answers`가 되었음

핵심

`answer` 메서드가 `self.answers`를 사용

클래스와 Chainlit 연결

bot 객체를 만들고, on_message에서 사용합니다.

```
import chainlit as cl

class AdviceBot:
    def __init__(self):
        self.answers = {
            "공부": "오늘 할 일을 작게 나눠보세요.",
            "진로": "좋아하는 과목부터 적어보세요."
        }

    def answer(self, user_text):
        for keyword, reply in self.answers.items():
            if keyword in user_text:
                return reply
        return "조금 더 자세히 말해줄래요?"

bot = AdviceBot()

@cl.on_message
async def on_message(message: cl.Message):
    reply = bot.answer(message.content)
    await cl.Message(content=reply).send()
```

연결 지점

```
reply =
bot.answer(message.content)
```

실행 결과

사용자는 화면에서 챗봇처럼 사용

해보기 4

AdviceBot 안의 답변 목록을 바꿉니다.

1. self.answers에 키워드 2개 추가하기
2. 기본 답변 문구를 바꾸기
3. 채팅창에서 새 키워드가 동작하는지 확인하기

```
"급식": "오늘 급식 메뉴를 확인해보세요.",  
"운동": "가볍게 걷기부터 시작해보세요."
```

`__init__`은 처음 준비하는 곳

객체가 만들어질 때 필요한 정보를 준비합니다.

```
class AdviceBot:
    def __init__(self, name):
        self.name = name
        self.answers = {
            "공부": "오늘 할 일을 작게 나눠보세요.",
            "진로": "좋아하는 과목부터 적어보세요."
        }

bot = AdviceBot("학교생활봇")
```

`self.name`

이 챗봇 객체의 이름

`bot`

AdviceBot 클래스로 만든 객체

self는 “지금 이 객체”를 가리킵니다

객체마다 자기 정보를 따로 가질 수 있습니다.

```
class AdviceBot:
    def __init__(self, name):
        self.name = name
        self.answers = {
            "공부": "오늘 할 일을 작게 나눠보세요.",
            "진로": "좋아하는 과목부터 적어보세요."
        }
```

```
bot1 = AdviceBot("공부봇")
```

```
bot2 = AdviceBot("진로봇")
```

```
print(bot1.name)
```

```
print(bot2.name)
```

bot1.name

공부봇

bot2.name

진로봇

같은 AdviceBot 클래스로 만들었지만,
bot1과 bot2는 서로 다른 객체

이제 이름까지 포함해 답하기

객체가 가진 name을 답변에 사용합니다.

```
class AdviceBot:
    def __init__(self, name):
        self.name = name
        self.answers = {
            "공부": "오늘 할 일을 작게 나눠보세요.",
            "진로": "좋아하는 과목부터 적어보세요."
        }

    def answer(self, user_text):
        for keyword, reply in self.answers.items():
            if keyword in user_text:
                return f"{self.name}: {reply}"
        return f"{self.name}: 조금 더 자세히 말해줄래요?"

bot1 = AdviceBot("공부봇")
bot2 = AdviceBot("진로봇")

print(bot1.answer("공부"))
```

결과 예시

공부봇: 오늘 할 일을 작게
나눠보세요.

해보기 5

챗봇 이름과 답변 스타일을 바꿔봅니다.

1. `bot = AdviceBot("우리팀봇")`으로 이름 바꾸기
2. 답변 앞에 챗봇 이름이 나오는지 확인하기
3. 기본 답변을 우리 팀 말투로 바꾸기

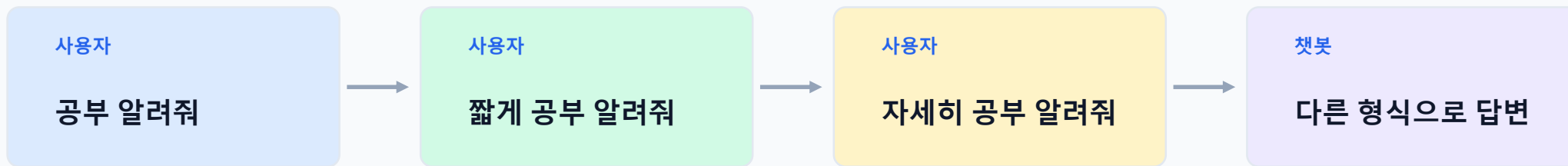
```
bot = AdviceBot("2팀 상담봇")
```

4. Chainlit에서 조금 더 챗봇답게 만들기

답변 길이, 시작 인사, 대화 횟수까지 화면에 반영합니다.

같은 질문도 답변 길이가 다를 수 있습니다

사용자가 “짧게” 또는 “자세히”라고 말하면 답변 방식을 바꿉니다.



이 기능을 위해 find_mode라는 메서드를 AdviceBot 클래스 안에 추가합니다. 혼자 떨어진 함수가 아니라, answer 메서드가 사용하는 보조 메서드입니다.

find_mode를 클래스 안에 추가하기

이 메서드는 답변 길이 모드를 판단합니다.

```
class AdviceBot:
    def find_mode(self, user_text):
        if "짧게" in user_text:
            return "short"
        elif "자세히" in user_text:
            return "detail"
        else:
            return "normal"
```

어디에 넣나요?

AdviceBot 클래스 안에 넣습니다.

누가 쓰나요?

answer 메서드가 호출해서 사용합니다.

answer에서 find_mode 사용하기

모드를 판단한 뒤 답변을 조금 다르게 만듭니다.

```
def answer(self, user_text):
    mode = self.find_mode(user_text)

    for keyword, reply in self.answers.items():
        if keyword in user_text:
            if mode == "short":
                return f"{self.name}: {reply}"
            elif mode == "detail":
                return f"{self.name}: {reply} 예를 들어 오늘 할 일 1개만 정해보세요."
            else:
                return f"{self.name}: {reply}"

    return f"{self.name}: 조금 더 자세히 말해줄래요?"
```

사용자

공부 알려줘

사용자

짧게 공부 알려줘

사용자

자세히 공부 알려줘

해보기 6

답변 모드를 하나 더 추가합니다.

1. "예시"가 들어오면 example 모드가 되게 하기
2. example 모드일 때 예시 문장을 붙이기
3. 짧게 / 자세히 / 예시 세 가지를 모두 테스트하기

```
elif "예시" in user_text:  
    return "example"
```

대화가 시작될 때 인사하기

Chainlit에는 채팅 시작 시점도 있습니다.

```
@cl.on_chat_start
async def on_chat_start():
    await cl.Message(
        content="안녕하세요. 학교생활봇입니다."
    ).send()
```

@

Chainlit에서는 데코레이터를 사용해 특정 상황에서 실행될 함수를 등록.

on_chat_start

채팅 화면이 처음 열릴 때 실행됩니다.

on_message

사용자가 메시지를 보낼 때 실행됩니다.

질문 횟수를 기억해보기

Chainlit의 `user_session`을 사용하면 대화 중 값을 저장할 수 있습니다.

```
@cl.on_chat_start
async def on_chat_start():
    cl.user_session.set("count", 0)
    await cl.Message(content="질문을 입력해보세요.").send()

@cl.on_message
async def on_message(message: cl.Message):
    count = cl.user_session.get("count")
    count = count + 1
    cl.user_session.set("count", count)

    reply = bot.answer(message.content)
    await cl.Message(content=f"{count}번째 질문입니다.{reply}").send()
```

무엇을 기억하나요?

현재 대화에서 질문을 몇 번
했는지 기억합니다.

해보기 7

질문 횟수를 화면에 표시합니다.

1. on_chat_start에서 count를 0으로 저장하기
2. on_message에서 count를 1씩 늘리기
3. 답변 앞에 “n번째 질문”이 보이면 성공

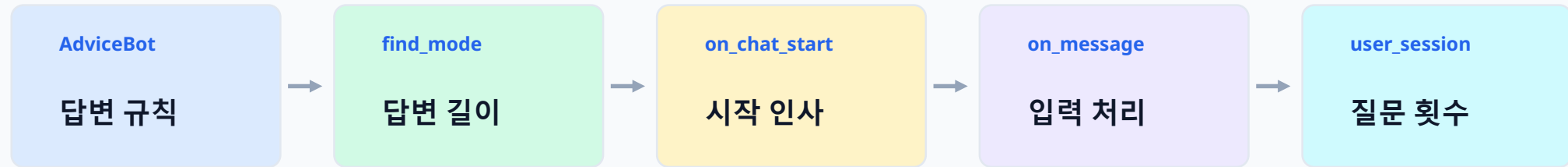
```
python -m chainlit run test/test.py -w
```

5. 나만의 챗봇 만들기

지금까지 만든 기능을 하나로 합쳐 실제 실습 결과물을 만듭니다.

최종 챗봇 구조

이제 코드가 여러 부분으로 나뉩니다.



이번 실습의 목표는 완벽한 AI가 아니라, Chainlit 화면에서 동작하는 클래스 기반 챗봇을 완성하는 것입니다.

최종 코드 1: 클래스 시작

챗봇 이름과 답변 목록을 준비합니다.

```
import chainlit as cl

class AdviceBot:
    def __init__(self, name):
        self.name = name
        self.answers = {
            "공부": "오늘 할 일을 작게 나눠보세요.",
            "진로": "좋아하는 과목부터 적어보세요.",
            "친구": "상대 이야기를 먼저 들어보세요."
        }
```

최종 코드 2: 모드와 답변

사용자 문장을 보고 답변 형식을 정합니다.

```
def find_mode(self, user_text):
    if "짧게" in user_text:
        return "short"
    elif "자세히" in user_text:
        return "detail"
    else:
        return "normal"

def answer(self, user_text):
    mode = self.find_mode(user_text)
    for keyword, reply in self.answers.items():
        if keyword in user_text:
            if mode == "detail":
                return f"{self.name}: {reply} 작은 예시부터 시작해보세요."
            return f"{self.name}: {reply}"
    return f"{self.name}: 조금 더 자세히 말해줄래요?"
```

최종 코드 3: Chainlit 연결

시작 인사와 메시지 처리를 연결합니다.

```
bot = AdviceBot("학교생활봇")

@cl.on_chat_start
async def on_chat_start():
    cl.user_session.set("count", 0)
    await cl.Message(content="안녕하세요. 질문을 입력해보세요.").send()

@cl.on_message
async def on_message(message: cl.Message):
    count = cl.user_session.get("count") + 1
    cl.user_session.set("count", count)

    reply = bot.answer(message.content)
    await cl.Message(content=f"{count}번째 질문입니다.{reply}").send()
```

해보기 8

나의 챗봇을 만듭니다.

1. 챗봇 이름 바꾸기
2. answers 키워드와 답변을 5개 만들기
3. 짧게 / 자세히 모드가 동작하는지 확인하기
4. 질문 횟수가 정상적으로 늘어나는지 확인하기

예시 주제: 진로 상담, 공부 계획, 급식 안내, 영어 단어 퀴즈, 학교생활 고민 상담

팀 주제 예시

키워드와 답변을 이렇게 바꿀 수 있습니다.

```
self.answers = {  
    "개발자": "개발자는 문제를 코드로 해결하는 일을 합니다.",  
    "디자인": "디자인은 사용자가 보기 쉽고 쓰기 쉽게 만드는 일입니다.",  
    "데이터": "데이터 분야는 숫자와 기록에서 의미를 찾는 일입니다.",  
    "보안": "보안은 시스템과 정보를 안전하게 지키는 일입니다.",  
    "진로": "좋아하는 과목과 오래 해도 덜 지치는 활동을 적어보세요."  
}
```

입력 예시

자세히 개발자 알려줘

입력 예시

짧게 보안 알려줘

디버깅 카드

오류가 나면 먼저 실행 방식과 들여쓰기를 확인합니다.

ModuleNotFoundError

python -m pip install chainlit

실행 버튼 오류

python -m chainlit run test/test.py -w

IndentationError

class / def / if 아래 들여쓰기 확인

NameError

변수 이름 철자 확인

화면이 안 열림

PORTS 탭에서 8000 열기

반영이 안 됨

ctrl + c : 터미널 중지 후 다시 실행

6. 다음 시간으로 이어지는 생각

규칙 기반 응답에서 생성형 기반 응답으로 확장 될수 있다면

규칙 기반 챗봇의 한계

지금 챗봇은 answers에 없는 내용은 잘 대답하지 못합니다.

규칙 기반 응답 방식

미리 정한 키워드와 답변을 찾습니다.

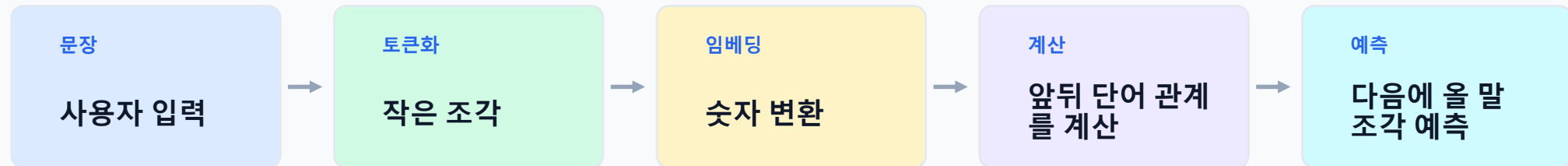
LLM 방식

사용자 문장을 보고 새 답변을 생성합니다.

그래서 실제 AI 서비스에서는 “사용자의 요청을 정리한 프롬프트”를 LLM에 보내고,
LLM이 답변을 생성하게 합니다.

다음 시간 미리보기

LLM은 문장을 어떻게 처리할까요?



오늘은 실제 LLM 내부를 깊게 배우지 않습니다. 대신 “문장이 여러 단계를 거쳐 다음 단어 예측으로 이어진다”는 흐름만 미리 봅니다.

다음 시간 맛보기

입력 문장이 단계별로 어떻게 바뀌는지 보여줍니다.

```
import chainlit as cl

class TextProcessPreview:
    def tokenize(self, text):
        return text.split()

    def to_numbers(self, tokens):
        return [len(token) for token in tokens]

    def make_preview(self, text):
        tokens = self.tokenize(text)
        numbers = self.to_numbers(tokens)

        return f"""
입력 문장:
{tokens}

1단계. 문장을 작은 조각으로 나누기:
{tokens}

2단계. 각 조각을 간단한 숫자로 바꿔보기:
{numbers}
"""

preview = TextProcessPreview()

@cl.on_message
async def on_message(message: cl.Message):
    result = preview.make_preview(message.content)

    await cl.Message(content=result).send()
```

다음 시간에는 실제 LLM이 이런 숫자 표현을 훨씬 더 복잡하게 사용해 문맥을 계산하고 다음 말을 예측하는 과정을 배웁니다.

오늘 기억할 세 문장

세 문장만 남아도 충분합니다.

1

오늘은 규칙대로 답하는 챗봇을 만들었습니다.

2

Chainlit을 쓰면 Python 코드로 채팅 화면을 만들 수 있습니다.

3

클래스는 객체를 만들기 위한 코드의 틀입니다.

작게 만들고, 조금씩 구조를 정리하면 됩니다.

오늘은 규칙 기반 챗봇을 만들었습니다.

다음 시간에는 LLM이 어떻게 규칙 없이도 문맥에 맞는 답변을 생성하는지 살펴봅니다.